

POO 3 (JAVA ET JAVA AVANCÉ)

Prof. Nisrine DAD

4° Ingénierie Informatique et Réseaux - Semestre 1

Ecole Marocaine des Sciences d'Ingénieur

Année Universitaire : 2024/2025

IV. EXPRESSIONS LAMBDA

IV. EXPRESSIONS LAMBDA

- Java est un langage orienté objet : à l'exception des instructions et des données primitives, tout le reste est objets, même les tableaux et les chaînes de caractères.
- Java ne propose pas la possibilité de définir une fonction/méthode en dehors d'une classe ni de passer une telle fonction en paramètre d'une méthode.
- **Depuis Java 1.1**, la solution pour passer des traitements en paramètres d'une méthode est d'utiliser les **classes anonymes internes**.

IV. EXPRESSIONS LAMBDA

Classe anonyme interne: Exemple

```
List<Integer> liste=Arrays.asList(1,2,7,1,6,4,1);  
// Affichage avec la boucle for  
for(int i=0;i<liste.size();i++)  
    System.out.println(liste.get(i));  
  
// Affichage avec la méthode forEach()  
liste.forEach(new Consumer<Integer>(){  
    @Override  
    public void accept(Integer t) {  
        System.out.println(t);  
    }  
});
```

1
2
7
1
6
4
1
1
2
7
1
6
4
1

IV. EXPRESSIONS LAMBDA

Classe anonyme interne: Exemple

Modifier and Type	Method and Description
default void	<code>forEach</code> (<code>Consumer</code> <? super <code>T</code> > action) Performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);

    default Consumer<T> andThen(Consumer<? super T> after) {
        Objects.requireNonNull(after);
        return (T t) -> { accept(t); after.accept(t); };
    }
}
```

IV. EXPRESSIONS LAMBDA

Interface fonctionnelle

- Une interface fonctionnelle (**functional interface**) est basiquement une interface dans laquelle **une seule méthode abstraite** est définie.
- Elle doit respecter certaines contraintes :
 - **elle ne doit avoir qu'une seule méthode déclarée abstraite**
 - **toutes les méthodes doivent être public**
 - **elle peut avoir des méthodes par défaut et static**

IV. EXPRESSIONS LAMBDA

Interface fonctionnelle

- Avant Java 8 un certain nombre d'interfaces ne définissaient qu'une seule méthode : ces interfaces sont désignées par le terme **Single Abstract Method (SAM)**.
- Avant Java 8, le JDK contenait déjà de nombreuses interfaces qui respectaient ces règles et sont donc des interfaces fonctionnelles, par exemple :
 - **Comparator<T>** qui définit la méthode `int compare(T o1, T o2)`
 - **Callable<V>** qui définit la méthode `V call() throws exception`
 - **Runnable** qui définit la méthode `void run()`
 - **ActionListener** qui définit la méthode `void actionPerformed(ActionEvent)`

IV. EXPRESSIONS LAMBDA

Comparable<T> vs Comparator<T>

- L'interface **Comparable** est utilisée pour définir un **ordre naturel** des objets d'une classe. Si une classe implémente l'interface Comparable, cela signifie que ses instances peuvent être comparées entre elles en utilisant la méthode compareTo.
- L'implémentation de la méthode compareTo est **spécifique à la classe** et définit la logique de comparaison entre les instances de cette classe.
- On peut trier une liste d'objets d'une classe implémentant Comparable en appelant **Collections.sort()** ou **utiliser des structures de données qui impliquent un ordre naturel**.

IV. EXPRESSIONS LAMBDA

Comparable<T> vs Comparator<T>

- L'interface **Comparator** est utilisée pour définir un **ordre externe** à une classe, c'est-à-dire un ordre qui n'est pas basé sur l'ordre naturel des objets.
- Cela permet de trier une liste d'objets en **fonction d'un critère spécifié** par le développeur.

```
import java.util.Comparator;
public class EtudiantParNote implements Comparator<Etudiant>{
    @Override
    public int compare(Etudiant o1, Etudiant o2) {
        return (int)o1.getMoyenne()-(int)o2.getMoyenne();
    }
}
```

```
import java.util.Comparator;
public class EtudiantParNom implements Comparator<Etudiant>{
    @Override
    public int compare(Etudiant o1, Etudiant o2) {
        return o1.getNom().compareTo(o2.getNom());
    }
}
```

```
Collections.sort(g1.getListe(),new EtudiantParNom());
g1.lister();
Collections.sort(g1.getListe(),new EtudiantParNote());
g1.lister();
```

IV. EXPRESSIONS LAMBDA

Exercice

- Est-ce-que les interfaces `Comparable<T>` et `Comparator<T>` sont des interfaces fonctionnelles? Justifiez votre réponse.
- Dans le slide 10, remplacez l'utilisation des classes externes par des classes anonymes internes pour trier la liste avec `Comparator<T>`.

IV. EXPRESSIONS LAMBDA

- Pour faciliter la mise à œuvre de la programmation fonctionnelle, Java 8 propose les expressions lambda.
- Les **expressions lambda** sont aussi nommées **closures** ou **fonctions anonymes** : leur but principal est de **permettre de passer en paramètre un ensemble de traitements**.
- Elle permet de définir une implémentation d'une interface fonctionnelle sous la forme d'une expression composée d'une liste de paramètres et d'un corps qui peut être une simple expression ou un bloc de code.

```
List<Integer> liste=Arrays.asList(1,2,7,1,6,4,1);  
liste.forEach(e->System.out.println(e));
```

IV. EXPRESSIONS LAMBDA

Package `java.util.function`

- Les interfaces définies avec des types génériques sont :
- **Consumer<T>** : opération qui accepte **un unique argument** (type T) et **ne retourne pas de résultat**. - **void accept(T);**
- **Function<T,R>** : opération qui **accepte un argument** (type T) et **retourne un résultat** (type R). - **R apply(T);**
- **Supplier<T>** : opération qui **ne prend pas d'argument** et qui **retourne un résultat** (type T). - **T get();**
- **Predicate<T>** est une spécialisation de Function visant à tester une valeur et retourner un booléen. - **boolean test(T);**

IV. EXPRESSIONS LAMBDA

Package java.util.function

- En plus, de nombreuses autres interfaces fonctionnelles sont définies avec des types de base : `IntConsumer`, `LongToIntFunction`, `DoubleSupplier`, etc.

```
Consumer<Integer> afficher = (e) -> System.out.println(e);  
afficher.accept(2);
```

```
Function<Integer, Integer> carre = a -> a*a;  
System.out.println(carre.apply(4));
```

```
IntFunction<Double> carreInt=a-> a/2.0;  
System.out.println(carreInt.apply(2));
```

```
BiFunction<Integer, Integer, Integer> additionnerFunction = (x, y) -> x + y;  
System.out.println(additionnerFunction.apply(2, 3));  
BiConsumer<Integer, Integer> additionner = (x, y) -> System.out.println(x+y);  
additionner.accept(200, 3);
```

IV. EXPRESSIONS LAMBDA

Package java.util.function

```
Function<Integer, Boolean> estPositif = a -> {  
    if(a>0)  
        return true;  
    else return false;  
};  
System.out.println(estPositif.apply(-2));
```

```
Predicate<Integer> estPositif1= a->a>0;  
System.out.println(estPositif1.test(2));
```

```
Supplier<Double> randomValue = () -> Math.random();  
System.out.println(randomValue.get());
```

IV. EXPRESSIONS LAMBDA

Références de méthode

- Les références de méthodes permettent d'offrir une syntaxe simplifiée pour invoquer une méthode comme une expression lambda :
- elles offrent un raccourci syntaxique pour créer une expression lambda dont le but est d'invoquer une méthode ou un constructeur.
- Une expression lambda correspond à une méthode anonyme dont le type est défini par une interface fonctionnelle. Les références de méthodes ont un rôle similaire mais au lieu de fournir une implémentation, une référence de méthode permet d'invoquer une méthode statique ou non ou un constructeur.
- Les références de méthodes ou de constructeurs utilisent le nouvel opérateur ::.

IV. EXPRESSIONS LAMBDA

Type	Référence de méthode	Expression lambda
Référence à une méthode statique	nomClasse::nomMethodeStatique	String::valueOf
Référence à une méthode sur une instance	objet::nomMethode	personne::toString
Référence à une méthode d'un objet arbitraire d'un type donné	nomClasse::nomMethode	Object::toString
Référence à un constructeur	nomClasse::new	Personne::new

Type	Exemples	Exemples
Référence à une méthode statique	System.out::println Math::pow	x -> System.out.println(x) (x, y) -> Math.pow(x, y)
Référence à une méthode sur une instance	monObject::maMethode	x -> monObject.maMethode(x)
Référence à une méthode d'un objet arbitraire d'un type donné	String::compareToIgnoreCase	(x, y) -> x.compareToIgnoreCase(y)
Référence à un constructeur	MaClasse::new	() -> new MaClasse();